Changing UFS gear and lanes at runtime on MT6991

Why the debugfs write failed with “exit h8 state fail”, and why one line fixed it. Explained from scratch.

Written 31 October 2026 · MediaTek MT6991 SoC evaluation board · Android GKI kernel 6.6 (`android15-6.6`) with MediaTek vendor modules

THE WHOLE STORY IN SIX LINES

1. The board's kernel has a debugfs file, `power_mode`. Writing six digits to it (for example `332211`) should change the speed (gear), width (lanes) and mode of the link between the processor and its UFS storage chip.
2. The write woke up the UFS *controller* inside the processor, but not the UFS *chip*. The link between them stayed in **hibern8**, its deep-sleep state.
3. Before any speed change, MediaTek's driver checks that the link is awake. It waited 100 ms, the link never woke, and it printed `exit h8 state fail`.
4. The fix was one call: `ufs_debugfs_get_user_access()`, which runs `ufshcd_rpm_get_sync()`, instead of `pm_runtime_get_sync(hba->dev)`. The new call wakes the chip side too.
5. The old line was correct in Linux 4.19, where the ported code came from. In Linux 5.14 (2021) the job of waking the link moved to a different object, so the same line now does less.
6. Result on the board, 31 October: no more failures, and lane changes work.

How to read this

Every idea starts with the physical thing (wires, chips, registers) before it gets a name. You'll see six kinds of boxes:

PLAIN WORDS	The simplest version of the idea.
TWO MEANINGS	A word that means different things in different places. These caused real confusion.
KEY POINT	The thing to remember from that section.
WATCH OUT	A trap you can fall into on the board.
CORRECTION	Something stated during the investigation and later corrected.
TRY IT	Commands to run on the board.

Line numbers come from the copies read for this write-up: MediaTek's vendor code from OnePlus's public MT6991 tree (`kernel_device_modules-6.6`), the Android common kernel `android15-6.6`, Qualcomm's msm-4.19 `ufs-debugfs.c`, and upstream Linux v5.13. The board's vendor tree has the same code but may be a few lines off.

Contents

1. The hardware: two chips and some wires 3
2. Sleep: what hibern8 is 7
3. The software: who's who in Linux 9
4. Why the original line was right once 14
5. What happened on the board 16
6. The fix, and the other changes 20
7. How we got here, and where we went wrong
8. How we know it's fixed 2725
9. Open items 29
10. Glossary 30
11. Files in this folder 32
12. References 33

1 The hardware: two chips and some wires

1.1 What is physically on the board

Two chips matter here.

- **The MT6991 processor (the SoC).** Besides the CPU cores, it contains a block of logic called the **UFS host controller**. In this document it's **the controller**.
- **The UFS storage chip.** NAND flash memory plus its own small processor that manages the flash. In this document it's **the chip**.

Copper traces on the circuit board connect the two. These traces are the **lanes**. A lane carries bits in one direction only. Each lane is a *pair* of wires, and a bit is sent as the voltage *difference* between them: one wire goes up while the other goes down. Noise that hits both wires equally cancels out, which is why this works at billions of bits per second. MT6991 uses two lanes from the controller to the chip (transmit, TX) and two lanes back (receive, RX). That's eight data wires. The standard that defines these wires and their voltages is called **M-PHY**.

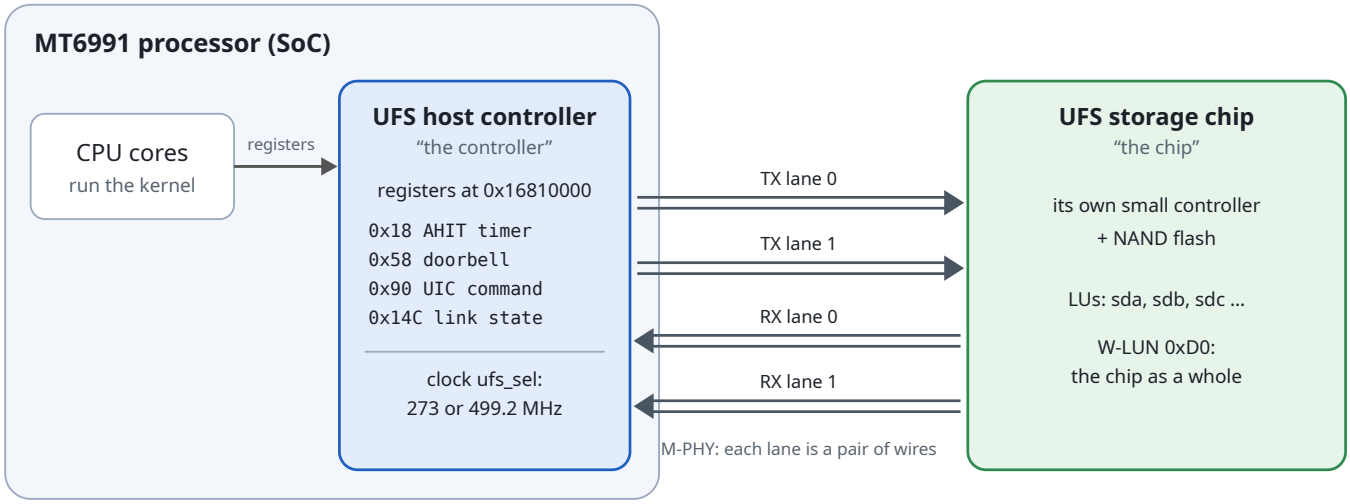


Figure 1. The controller is inside the SoC; the chip is a separate package. The CPU never touches the lanes. It only reads and writes the controller's registers, and the controller drives the wires.

The CPU talks to the controller through **registers**. A register is a memory address wired to hardware instead of to RAM. Writing a number there makes the hardware do something; reading it tells you what the hardware is doing. The controller's registers start at physical address `0x16810000`, the `ufshci@16810000` node in the device tree (`mt6991.dts:6686`). Four of them matter in this story (offsets from that base):

OFFSET	NAME	WHAT IT DOES
`0x18`	AHIT (auto-hibernate idle timer)	How long the link may sit idle before the controller puts it to sleep by itself. Writing 0 turns this off. (Part 2.)
`0x58`	Transfer request doorbell	One bit per command slot. The driver sets a bit to say "slot *n* has a command, go"; the controller clears it when the command is done.

OFFSET	NAME	WHAT IT DOES
`0x90`	UIC command	Where the driver writes commands for the link itself: wake it, put it to sleep, read or change a link setting.
`0x14C`	MediaTek link state (`REG_UFS_UFS_MMIO_OTSD_CTRL`)	MediaTek-specific. Its top byte says whether the link is up, down or asleep. (Part 2.)

> **TWO MEANINGS**
>
> **"Host"** can mean the controller hardware, Linux's SCSI host object (`struct Scsi_Host`), or loosely the whole phone. **"Device"** can mean the UFS chip, or any `struct device` in Linux's tree of objects. In this document, *the controller* and *the chip* are always the hardware. A plain "device" means the Linux object.

1.2 Three layers of rules on the wires

Wires alone aren't enough; both ends need rules. UFS stacks three sets of them, and the chip runs the same stack on its side:

UFS transport (UTP)	Carries SCSI commands (READ, WRITE, START STOP UNIT, …) in packets called UPIUs. This is what the files are made of.
UniPro	The link layer. Packs bits into frames, checks them, acknowledges them and re-sends anything lost. It also owns the link's *speed and power state*. Its settings are numbered **attributes**, such as `PA_TXGEAR` (0x1568, the transmit gear) or `PA_ACTIVETXDATALANES` (0x1560, how many transmit lanes are in use). "PA" stands for PHY Adapter, the bottom part of UniPro.
M-PHY	The electrical layer: drives the voltages, recovers bits and timing at the other end, puts the wires to sleep and wakes them.

The driver reaches UniPro through **UIC commands**, written to register 0x90. The ones in this story are `DME_GET` and `DME_SET` (read or write one attribute), and `DME_HIBERNATE_ENTER` and `DME_HIBERNATE_EXIT` (put the link to sleep, wake it up). DME is the part of UniPro that accepts these commands (the Device Management Entity).

> **PLAIN WORDS**
>
> SCSI is what the letter says. UniPro is the delivery service that numbers the letters, confirms delivery and re-sends lost ones. M-PHY is the road.

1.3 Gears: how fast the wires flip

A **gear** is how many bits per second each lane carries. The high-speed (HS) gears go from G1 to G5, and each gear doubles the one before. There are two slightly different base frequencies, called **rate A** and **rate B**. MediaTek always uses rate B (it's the default, `PA_HS_MODE_B`, in the core's shared negotiation helper).

GEAR	BITS PER SECOND PER LANE (RATE B)	ONE BIT LASTS
HS-G1	1,457.6 million	686 ps
HS-G2	2,915.2 million	343 ps
HS-G3	5,830.4 million	172 ps
HS-G4	11,660.8 million	86 ps
HS-G5	23,321.6 million	43 ps

To make that physical: at G5 one bit lasts 43 picoseconds (trillionths of a second). In that time a signal travelling along a circuit-board trace covers about 6 or 7 millimetres.

Not every bit on the wire is your data. Gears G1 to G4 send every 8 bits of data as 10 bits on the wire (**8b/10b** encoding), which keeps the signal balanced and lets the receiver stay locked to its timing. G5 uses the more efficient 128b/130b. So the data a lane can carry is its bit rate × 0.8 (× 128/130 at G5). Part 8 uses this to check that a gear change really happened.

There is also a slow family, the **PWM gears**, used for bring-up and low power. They run from a few megabits per second to a few hundred.

The controller's own clock has to keep up with the gear. On MT6991 that clock is `ufs_sel`, which switches between a 273 MHz source and a 499.2 MHz source (`freq-table-hz = <273000000 499200000>`, `mt6991.dts:6732`). MediaTek's code pairs the top gear with 499.2 MHz and G4 with 273 MHz (section 6.4). The 499.2 MHz source needs the SoC's core supply voltage (vcore) at 0.65 V or more (`clk-scale-up-vcore-min = <650000>`, `mt6991.dts:6751`).

1.4 Lanes

Lanes are parallel roads. With two lanes the data is split across both, so the link carries twice as much. Each direction can use 1 or 2 lanes.

> **TWO MEANINGS**
>
> “Two lanes” in UFS means two *per direction*: 2 TX and 2 RX, four wire pairs. In the six digits, the lane part `22` is one digit for RX and one for TX.

1.5 Modes: the last two digits

- `1` = FAST_MODE: the HS gears.
- `2` = SLOW_MODE: the PWM gears.
- `4` = FASTAUTO, `5` = SLOWAUTO: the same, except that the link may drop into a lower-power state on its own between bursts of data.

TWO MEANINGS: “POWER MODE” HAS THREE

1. **Link power mode** (UniPro): FAST, SLOW, FASTAUTO, SLOWAUTO. The link's speed setting. The digits set this.
2. **Chip power mode**: Active, Sleep, PowerDown. Set by sending the chip the SCSI command START STOP UNIT (often shortened to SSU).
3. **Linux runtime PM state** of a `struct device`: active or suspended (Part 3).

The bug involved all three at once.

1.6 A power mode change, step by step

Changing gear, lanes or mode is a **power mode change**, or **PMC**. Both ends must switch at the same moment, so a PMC is a conversation over the link:

1. The driver writes the new settings into the controller's UniPro attributes: `PA_RXGEAR`, `PA_ACTIVERXDATALANES`, `PA_RXTERMINATION`, `PA_TXGEAR`, `PA_ACTIVETXDATALANES`, `PA_TXTERMINATION`, `PA_HSSERIES` (rate A or B), and some timeout values (`PA_PWRMODEUSERDATA0` and onwards).
2. It writes `PA_PWRMODE`. That write is the trigger.
3. UniPro negotiates the change with the chip, over the link.
4. The controller raises an interrupt with the result. “Local success” means done.

In the 6.6 code, `ufshcd_change_power_mode()` (`ufshcd.c:4560`) does steps 1 and 2, and `ufshcd_uic_pwr_ctrl()` (`ufshcd.c:4199`) waits up to 500 ms (`UIC_CMD_TIMEOUT_DEFAULT`) for step 4. If nothing arrives, it prints `pwr ctrl cmd 0x… with mode 0x… completion timeout` (`ufshcd.c:4237`).

KEY POINT

A PMC is a conversation over the link. If the link is asleep, nobody answers.

The six digits are `GGLLMM`: gear RX, gear TX, lanes RX, lanes TX, mode RX, mode TX. So `332211` means HS-G3, two lanes each way, FAST mode.

2 Sleep: what hibern8 is

2.1 The link's deep sleep

When there's nothing to send, keeping eight wires switching billions of times a second wastes power. **Hibern8** (say “hibernate”) is the link's deepest sleep that still remembers its settings. The transmitters stop driving the wires, and the circuits that generate the fast clocks (the PLLs) stop. The gear and lane settings are kept, so waking up is much quicker than starting the link from scratch.

> **TWO MEANINGS**
>
> Hibern8 is not the PC kind of hibernate, where RAM is saved to disk. Only the link sleeps; the phone stays on.

2.2 Two ways in, and one way out of each

Auto-hibern8 (AH8): the hardware does it. You put an idle time into the AHIT register (offset 0x18). If no command arrives for that long, the controller puts the link to sleep by itself, and when a new command arrives it wakes the link by itself. Software takes no part. MediaTek sets the idle time according to who made the chip: 3.5 ms for Samsung, 2 ms for Micron, 1 ms for everyone else (`ufs_mtk_fix_ahit()`, `ufs-mediatek.c:1488`). Writing 0 to AHIT turns the timer off.

Software hibern8: the driver does it. The driver sends the UIC command `DME_HIBERNATE_ENTER`. The link then stays asleep until the driver sends `DME_HIBERNATE_EXIT`. The hardware will not wake it on its own, and changing AHIT doesn't touch it.

> **PLAIN WORDS**
>
> Auto-hibern8 is a motion-sensor light: it goes off when nobody's around and comes back on when someone walks in. Software hibern8 is a light somebody switched off by hand: it stays off until somebody switches it back on.

> **KEY POINT**
>
> The whole bug in one sentence: MediaTek's check treated a hand-switched light as if it were a motion-sensor light, and waited for it to come on by itself.

2.3 How the driver sees whether the link is awake

MediaTek's controller reports the link's state in register `0x14C` (`REG_UFS_UFS_MMIO_OTSD_CTRL`, `ufs-mediatek.h:52`). Its top byte, bits 31 to 24, holds the state:

VALUE	NAME IN THE CODE	MEANING
0	`VS_LINK_DISABLED`	Link not started

VALUE	NAME IN THE CODE	MEANING
1	`VS_LINK_DOWN`	Link down
2	`VS_LINK_UP`	Awake and working
3	`VS_LINK_HIBERN8`	Asleep in hibern8
4	`VS_LINK_LOST`	Connection lost
5	`VS_LINK_CFG`	Being configured

ufs-mediatek.h:134

`ufs_mtk_wait_link_state()` (`ufs-mediatek.c:562`) reads that byte every 100 to 200 µs until it shows the state it wants, or gives up when its time limit runs out. Bits 20 to 16 of the same register show the controller's internal state machine; MediaTek's idle wait uses them to spot a hibern8 entry or exit that's still in progress (values 8 to 13).

```
static int ufs_mtk_wait_link_state(struct ufs_hba *hba, u32 state,
                                   unsigned long max_wait_ms)
{
        ...
        timeout = ktime_add_ms(ktime_get(), max_wait_ms);
        do {
                time_checked = ktime_get();
                if (host->legacy_ip_ver || host->ip_ver < IP_VER_MT6899) { ... older chips ... }
                else {
                        val = ufshcd_readl(hba, REG_UFS_UFS_MMIO_OTSD_CTRL);
                        val = val >> 24;
                }
                if (val == state)
                        return 0;
                usleep_range(100, 200);   /* Sleep for max. 200us */
        } while (ktime_before(time_checked, timeout));

        return -ETIMEDOUT;
}
```

ufs-mediatek.c:562 (trimmed)

MT6991 uses this register because its IP version (`0x10460000` or `0x10470000`) is at least `IP_VER_MT6899`. Older MediaTek chips read `REG_UFS_PROBE` instead. An earlier explanation called it `REG_UFS_PROBE`; that's the older chips' register.

3 The software: who's who in Linux

3.1 Core and vendor: where the ported code runs

On Android, Google builds one generic kernel image for every phone, the **GKI** (Generic Kernel Image). The UFS **core** is part of it: `drivers/ufs/core/ufshcd.c`, `ufs-debugfs.c` and their friends. MediaTek's UFS code (`ufs-mediatek.c`, `ufs-mediatek-sysfs.c`, …) is a **vendor module** loaded on top, from `kernel_device_modules`.

The core calls into the vendor module through **vops** ("variant operations"): a table of function pointers that MediaTek fills in. The ones in this story are:

- `pwr_change_notify`, called just before a PMC (`PRE_CHANGE`) and just after it (`POST_CHANGE`);
- `suspend` and `resume`, called when the chip side goes to sleep or wakes;
- `clk_scale_notify`, called around clock changes.

The debugfs file lives in the core's `ufs-debugfs.c`. It can call core functions, but it can't see MediaTek's private data (`struct ufs_mtk_host`). That's why the fix had to use a core helper. It's also why a change there needs a new kernel image, not just a new vendor module.

> **TWO MEANINGS: DEBUGFS AND SYSFS**
>
> Both are files the kernel invents on the fly. **sysfs** (`/sys`) is the stable interface for real users; MediaTek's `clkscale_control` lives there. **debugfs** (`/sys/kernel/debug`) is for developers and can change at any time; the `power_mode` file lives there.

3.2 The objects Linux creates for UFS

Linux keeps a tree of `struct device` objects, one for each thing it manages. For UFS the tree looks like this:

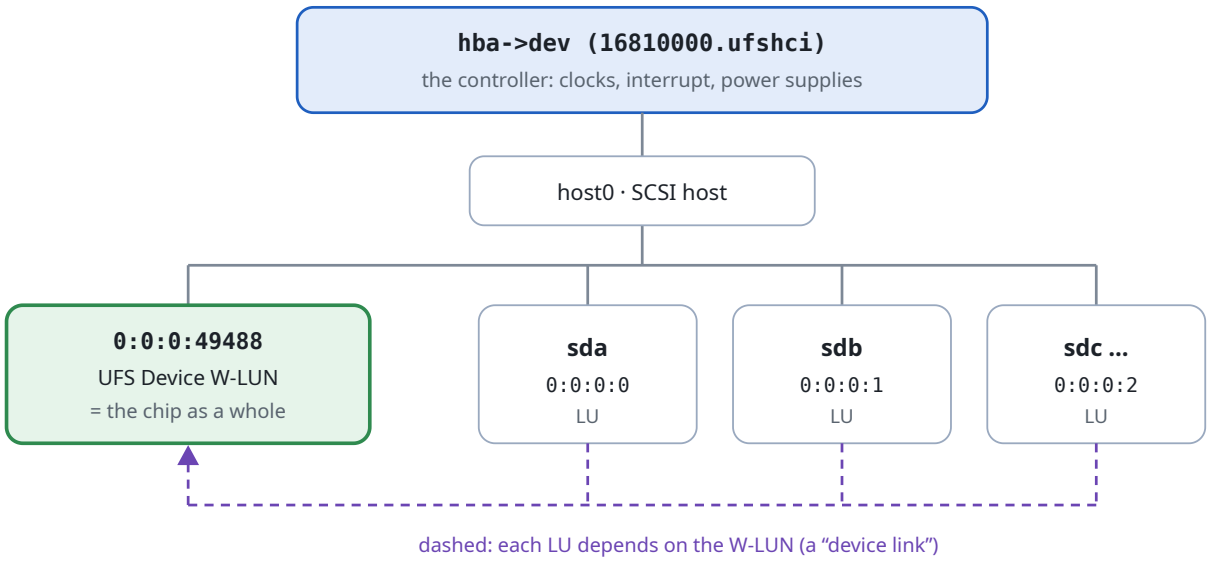


Figure 2. Solid lines are parent and child; dashed lines are "depends on". The W-LUN is Linux's handle on the chip itself. There are two more W-LUNs (RPMB and BOOT) that don't matter here.

- `hba->dev` is the controller, as a "platform device" named `16810000.ufshci`. Its jobs: the controller's clocks, interrupt and power supplies. (`hba` stands for host bus adapter, the core's `struct ufs_hba` for one controller.)
- `host0` is Linux's SCSI host. The chip speaks SCSI, so Linux handles it like a SCSI disk enclosure.
- **Logical units (LUs)** are slices of the chip that you can read and write. They appear as `sda`, `sdb`, `sdc`, and so on; your partitions live inside them.
- **Well-known logical units (W-LUNs)** are special addresses for jobs that concern the whole chip. The important one is the **UFS Device W-LUN**: to put the whole chip to sleep or wake it, you send START STOP UNIT to this address.

The UFS Device W-LUN is number `0xD0` in UFS's own numbering. Linux shows it as `0:0:0:49488`. The arithmetic: clear the W-LUN flag bit (`0x80`), which leaves `0x50`; add SCSI's base for well-known units, `0xC100`; that gives `0xC150`, which is 49488. (The RPMB W-LUN, `0xC4`, becomes 49476, and the BOOT W-LUN, `0xB0`, becomes 49456.)

> **PLAIN WORDS**
>
> The W-LUN is Linux's handle on *the chip itself*. Waking the W-LUN means waking the chip and the link that leads to it.

3.3 Runtime PM from scratch

Runtime PM is how Linux lets single parts of a running phone sleep while everything else keeps working. Every `struct device` has a usage counter and a state, *active* or *suspended*.

- `pm_runtime_get_sync(dev)` means "I need this device now". It adds 1 to the counter. If the device is asleep, it runs the device's wake-up function (its *resume callback*) and waits for it to finish.
- `pm_runtime_put(dev)` means "I'm done". It subtracts 1. At zero, the device may go to sleep after its **autosuspend delay**.

Four rules decide who wakes whom. Each has a physical reason:

1. **Waking a device wakes its parent first.** You can't reach the chip if the controller's clocks are off.
2. **Waking a parent never wakes its children.** Powering the controller doesn't mean anyone needs the chip.
3. **A parent can't sleep while a child is awake.**
4. **Device links add "depends on" arrows.** Each LU depends on the W-LUN (`device_link_add(…, DL_FLAG_PM_RUNTIME | DL_FLAG_RPM_ACTIVE)`, `ufshcd.c:5093`). So the W-LUN can't sleep while any LU is awake, and waking an LU wakes the W-LUN first.

On the board the LUs wait for 500 ms of quiet before they sleep (`MTK_RPM_AUTOSUSPEND_DELAY_MS`, `ufs-mediatek.h:367`, set as the SCSI host's `rpm_autosuspend_delay` at `ufs-mediatek.c:1769`), and the W-LUN waits 0 ms (`ufshcd_wl_probe()`, `ufshcd.c:10815`). So about half a second after the last disk access, the whole chip side is asleep.

TWO MEANINGS

Runtime suspend (one device sleeps while the phone is on) is not **system suspend** (the whole phone sleeps). This bug is entirely about runtime suspend.

And the one that caused confusion: `ufshcd_rpm_get_sync()` and `pm_runtime_get_sync()` are not two different mechanisms. The first is a one-line wrapper around the second:

```
static inline int ufshcd_rpm_get_sync(struct ufs_hba *hba)
{
        return pm_runtime_get_sync(&hba->ufs_device_wlun->sdev_gendev);
}
```

ufshcd-priv.h:344

Same function, different device: the W-LUN instead of `hba->dev`. Which device you name decides what wakes up.

3.4 What each object does when it sleeps and wakes (6.6)

This is where the real work happens. The code is short; read it. Comments starting with `←` are mine.

The W-LUN going to sleep: `__ufshcd_wl_suspend()`

```
        ret = ufshcd_vops_suspend(hba, pm_op, PRE_CHANGE);      // ← vendor hook
        ...
        if (req_dev_pwr_mode != hba->curr_dev_pwr_mode) {
                ...
                        ret = ufshcd_set_dev_pwr_mode(hba, req_dev_pwr_mode);
                                                // ← START STOP UNIT: chip to Sleep
        }
        ...
        ret = ufshcd_link_state_transition(hba, req_link_state, check_for_bkops);
                                                // ← DME_HIBERNATE_ENTER: link to hibern8
        ...
vops_suspend:
        ret = ufshcd_vops_suspend(hba, pm_op, POST_CHANGE);     // ← vendor hook
```

ufshcd.c:9746, lines 9830–9879 (trimmed)

How deep it sleeps is set by `rpm_lvl`. The core's default is level 3: chip in Sleep, link in hibern8 (`ufshcd.c:10676`). MediaTek switches to level 1 (chip stays Active, link in hibern8) only if the device tree has `mediatek,ufs-vcc-always-on`, which the MT6991 tree used here doesn't (`ufs-mediatek.c:2878`). Either way, **the link ends up in software hibern8**.

The W-LUN waking up: `__ufshcd_wl_resume()`

```
static int __ufshcd_wl_resume(struct ufs_hba *hba, enum ufs_pm_op pm_op)
{
        ...
        ret = ufshcd_vops_resume(hba, pm_op);                    // ← vendor hook
        ...
        if (ufshcd_is_link_hibern8(hba)) {
                ret = ufshcd_uic_hibern8_exit(hba);          // ← DME_HIBERNATE_EXIT
                if (!ret)
                        ufshcd_set_link_active(hba);
                ...
        } else if (ufshcd_is_link_off(hba)) {
                ret = ufshcd_reset_and_restore(hba);         // ← link was off: full restart
                ...
        }
        if (!ufshcd_is_ufs_dev_active(hba)) {
                ret = ufshcd_set_dev_pwr_mode(hba, UFS_ACTIVE_PWR_MODE);
                                                // ← START STOP UNIT: chip to Active
                ...
        }
        ...
        /* Enable Auto-Hibernate if configured */
        ufshcd_auto_hibern8_enable(hba);                         // ← AHIT timer back on
        ...
}
```

ufshcd.c:9927 (trimmed)

The controller waking up: `ufshcd_resume()`

```
static int ufshcd_resume(struct ufs_hba *hba)
{
        ...
        ufshcd_hba_vreg_set_hpm(hba);                // ← power supplies to full power
        ret = ufshcd_vreg_set_hpm(hba);
        ...
        /* Make sure clocks are enabled before accessing controller */
        ret = ufshcd_setup_clocks(hba, true);        // ← clocks on
        ...
        /* enable the host irq as host controller would be active soon */
        ufshcd_enable_irq(hba);                      // ← interrupt on

        goto out;
        ...
}
```

ufshcd.c:10163 (trimmed)

That's all the controller's wake-up does: power supplies, clocks, interrupt. Nothing about the link, nothing about the chip.

> **KEY POINT**
>
> In 6.6, the only wake-up that sends `DME_HIBERNATE_EXIT` is the W-LUN's. Waking the controller alone leaves the link asleep.

3.5 The sleep and wake order on the board

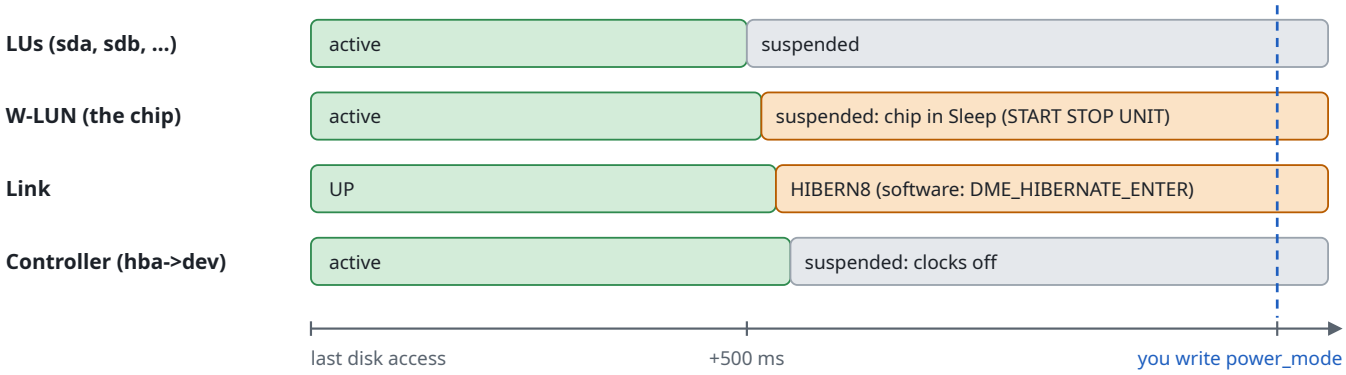


Figure 3. The sleep cascade. The LUs sleep after 500 ms of quiet; the W-LUN follows at once (chip to Sleep, link to software hibern8); then the controller turns its clocks off. By the time a command arrives, everything is asleep.

Going to sleep (500 ms without disk access): the LUs sleep → the W-LUN sleeps at once (chip to Sleep, link into software hibern8) → the controller sleeps (clocks off).

Waking with `ufshcd_rpm_get_sync()`, which names the W-LUN: rule 1 wakes the controller first (clocks on). Then the W-LUN's own wake-up runs: `DME_HIBERNATE_EXIT`, chip to Active, AH8 timer back on. The link is UP.

Waking with `pm_runtime_get_sync(hba->dev)`, which names the controller: clocks on. Then, by rule 2, nothing else. The link stays in hibern8 and the chip stays in Sleep.

4 Why the original line was right once

Was this “decided in 6.6”? It wasn't. It changed in Linux 5.14, in 2021, and 6.6 inherited it. the ported code comes from Qualcomm's Linux 4.19, from before the change.

4.1 Up to Linux 5.13: one switch did everything

Before 5.14, waking `hba->dev` ran `ufshcd_runtime_resume()`, which called `ufshcd_resume(hba, UFS_RUNTIME_PM)`. In v5.13 that single function did all of it:

```
static int ufshcd_resume(struct ufs_hba *hba, enum ufs_pm_op pm_op)
{
        ...
        ret = ufshcd_setup_clocks(hba, true);                   // 8860 ← clocks on
        ...
        ufshcd_enable_irq(hba);                                 // 8865 ← interrupt on
        --------- 6.6 cuts the function here ---------
        ret = ufshcd_vops_resume(hba, pm_op);                   //      ← vendor hook
        ...
        if (ufshcd_is_link_hibern8(hba)) {
                ret = ufshcd_uic_hibern8_exit(hba);             // 8880 ← DME_HIBERNATE_EXIT
        ...
        if (!ufshcd_is_ufs_dev_active(hba)) {
                ret = ufshcd_set_dev_pwr_mode(hba, UFS_ACTIVE_PWR_MODE);
                                                                // 8905 ← chip to Active
        ...
        ufshcd_auto_hibern8_enable(hba);                        // 8928 ← AHIT timer back on
        ...
}
```

upstream v5.13, drivers/scsi/ufs/ufshcd.c:8846 (trimmed; numbers are line numbers)

So in 4.19, `pm_runtime_get_sync(hba->dev)` woke the controller, the link and the chip. Qualcomm's msm-4.19 debugfs function starts with exactly that line (`ufs-debugfs.c:1211` in msm-4.19, copied in file 04), and the ported code is a port of that function.

4.2 The deadlock that moved the job

The change is commit `b294ff3e3449`, “scsi: ufs: core: Enable power management for wlun”, by Asutosh Das and Can Guo (both at Qualcomm). It was committed on 10 May 2021 and first released in Linux 5.14. Its commit message explains what went wrong before it:

1. When the controller went to sleep, its own sleep function sent START STOP UNIT to the chip, to put the chip to sleep first.
2. But by rule 3, the controller only sleeps after all its children are asleep, and that includes the SCSI devices. Their request queues were already suspended.
3. A block-layer change, `52abca64fd94` (“scsi: block: Do not accept any requests while suspended”), made any request to a suspended queue wait until the queue resumes.
4. So the START STOP UNIT sat waiting for a queue that would never resume, and the controller's sleep waited for the command. Nothing could move. The commit message shows the stuck call chain:

`ufshcd_runtime_suspend → ufshcd_suspend → ufshcd_set_dev_pwr_mode → __scsi_execute → blk_get_request → blk_queue_enter → schedule`.

The fix gave the W-LUN its own runtime PM and moved the chip-and-link work into *its* sleep and wake functions. With the device links, the W-LUN sleeps after all the LUs and wakes after the controller, so its START STOP UNIT always has a working path.

4.3 The proof: the old function, cut in two

Put v5.13's `ufshcd_resume()` next to 6.6 and cut it right after `ufshcd_enable_irq()`. The top half is 6.6's `ufshcd_resume()`, the controller's wake-up. The bottom half is 6.6's `__ufshcd_wl_resume()`, the W-LUN's wake-up. The lines are nearly identical, as you can check against the excerpts in section 3.4.

STEP	V5.13: ALL IN `UFSHCD_RESUME()`, RUN WHEN `HBA->DEV` WAKES	6.6: WHERE IT MOVED
Power supplies to full power	yes	`ufshcd_resume()`, the controller
Clocks on	line 8860	`ufshcd_resume()`, the controller
Interrupt on	line 8865	`ufshcd_resume()`, the controller
Vendor resume hook	yes	`__ufshcd_wl_resume()`, the W-LUN
`DME_HIBERNATE_EXIT`	line 8880	`__ufshcd_wl_resume()`, the W-LUN
Chip to Active (START STOP UNIT)	line 8905	`__ufshcd_wl_resume()`, the W-LUN
AHIT timer back on	line 8928	`__ufshcd_wl_resume()`, the W-LUN

LINUX 4.19 TO 5.13

`pm_runtime_get_sync(hba->dev)` wakes the controller, the link and the chip.

LINUX 5.14 AND LATER, INCLUDING 6.6

`pm_runtime_get_sync(hba->dev)` wakes the controller only.

PLAIN WORDS

In the old house, the main switch also turned on the hallway light. In 2021 the house was rewired and the hallway got its own switch. The ported instructions were written for the old house: “flip the main switch”. Flipping it left the hallway dark.

5 What happened on the board

5.1 The ported code path, start to finish

This is the ported code as of 31 October. Writing `echo 332211 > …/power_mode` runs:

1. **`ufsdbg_power_mode_write()`** reads the six digits, keeps the current rate (A or B), checks them with `ufsdbg_power_mode_validate()`, and compares them with the chip's maximum in `ufsdbg_cfg_pwr_param()`.
2. **`ufsdbg_config_pwr_mode()`** then:
   a. **Gets access to the hardware.** Originally with `pm_runtime_get_sync(hba->dev)`; now with `ufs_debugfs_get_user_access(hba)`.
   b. **`ufshcd_hold()`** keeps the controller's clocks on. Apart from runtime PM, the core also switches the controller's clocks off during short idle gaps. This is called **clock gating**, and on MediaTek it kicks in after at least 10 ms (`clk_gating.delay_ms = max(ah_ms, 10U)`). “Hold” blocks it while the code works.
   c. **`down_write(&hba->clk_scaling_lock)`** keeps the core's clock scaling, its error handler and its internal management commands (queries to the chip) from running at the same time. It's 6.6's replacement for msm-4.19's `hba->lock`.
   d. **`ufshcd_scsi_block_requests()`** tells the SCSI layer to stop sending new reads and writes.
   e. **`ufshcd_wait_for_doorbell_clr()`** waits until every command already sent has finished, that is, until the doorbell register (0x58) reads all zeros.
   f. **The ported clock lines** (section 6.3) and **`ufshcd_config_pwr_mode()`**, which runs MediaTek's `PRE_CHANGE` hook, then the PMC, then the `POST_CHANGE` hook.
   g. **Undo everything** in reverse order.

5.2 Error 1: the PMC timed out

This came from the first version, which, like Qualcomm's original (it calls `ufshcd_change_power_mode()` directly), sent the PMC without going through MediaTek's hooks. Whenever the board had been quiet for more than half a second, the W-LUN was asleep and the link was in software hibern8. The first line woke only the controller. The PMC was sent into a sleeping link, nothing answered, and after 500 ms the core gave up. It was first reported as “pmc command timed out”; the 6.6 core's wording is `pwr ctrl cmd … completion timeout`.

To be honest about this one: we never captured its full log. The sleeping link is the explanation that fits everything else, but it wasn't proven separately.

5.3 Error 2: “exit h8 state fail”

Next the call was routed through `ufshcd_config_pwr_mode()`, the way MediaTek's own `clkscale_control` path does. That runs MediaTek's `PRE_CHANGE` hook *before* the PMC:

```
static int ufs_mtk_pwr_change_notify(struct ufs_hba *hba,
                                     enum ufs_notify_change_status stage,
                                     struct ufs_pa_layer_attr *dev_max_params,
                                     struct ufs_pa_layer_attr *dev_req_params)
{
        int ret = 0;
        static u32 reg;

        switch (stage) {
        case PRE_CHANGE:
                if (ufshcd_is_auto_hibern8_supported(hba)) {
                        reg = ufshcd_readl(hba, REG_AUTO_HIBERNATE_IDLE_TIMER);
                        ufs_mtk_auto_hibern8_disable(hba);    // ← return value ignored
                }
                ret = ufs_mtk_pre_pwr_change(hba, dev_max_params,
                                             dev_req_params);
                break;
        case POST_CHANGE:
                if (ufshcd_is_auto_hibern8_supported(hba))
                        ufshcd_writel(hba, reg, REG_AUTO_HIBERNATE_IDLE_TIMER);
                break;
        ...
```

ufs-mediatek.c:2023

And the function it calls:

```
int ufs_mtk_auto_hibern8_disable(struct ufs_hba *hba)
{
        unsigned long flags;
        int ret;

        /* disable auto-hibern8 */
        spin_lock_irqsave(hba->host->host_lock, flags);
        ufshcd_writel(hba, 0, REG_AUTO_HIBERNATE_IDLE_TIMER);
        spin_unlock_irqrestore(hba->host->host_lock, flags);

        /* wait host return to idle state when auto-hibern8 off */
        ufs_mtk_wait_idle_state(hba, 5);

        ret = ufs_mtk_wait_link_state(hba, VS_LINK_UP, 100);
        if (ret) {
                dev_warn(hba->dev, "exit h8 state fail, ret=%d\n", ret);

                spin_lock_irqsave(hba->host->host_lock, flags);
                hba->force_reset = true;
                hba->ufshcd_state = UFSHCD_STATE_EH_SCHEDULED_FATAL;
                schedule_work(&hba->eh_work);
                spin_unlock_irqrestore(hba->host->host_lock, flags);

                /* trigger error handler and break suspend */
                ret = -EBUSY;
        }

        return ret;
}
```

ufs-mediatek.c:2997

Step by step:

1. **Save AHIT and write 0**, which switches auto-hibern8 off. MediaTek's code expects that switching the timer off makes the controller bring the link out of *auto*-hibern8 by itself. That's why the next steps only wait and

watch.

2. **Wait up to 5 ms** for any hibern8 entry or exit still in progress to finish.
3. **Poll register 0x14C for up to 100 ms**, waiting for the link state to read UP (2).
4. **It never did.** So it prints `exit h8 state fail, ret=-110` (−110 is `-ETIMEDOUT`), sets `force_reset`, marks the controller's state `UFSHCD_STATE_EH_SCHEDULED_FATAL` (3), schedules the error handler (which will reset the controller and the link) and returns `-EBUSY`.
5. **The caller ignores that return value** and goes on to `ufs_mtk_pre_pwr_change()` and then the PMC, on a link that's still asleep.

Why the link never came UP: it wasn't in auto-hibern8. The W-LUN's sleep had put it into *software* hibern8 with `DME_HIBERNATE_ENTER`. Switching the AHIT timer off can't undo that. Only `DME_HIBERNATE_EXIT` can, and the only code that sends it is the W-LUN's wake-up, which nothing had run.

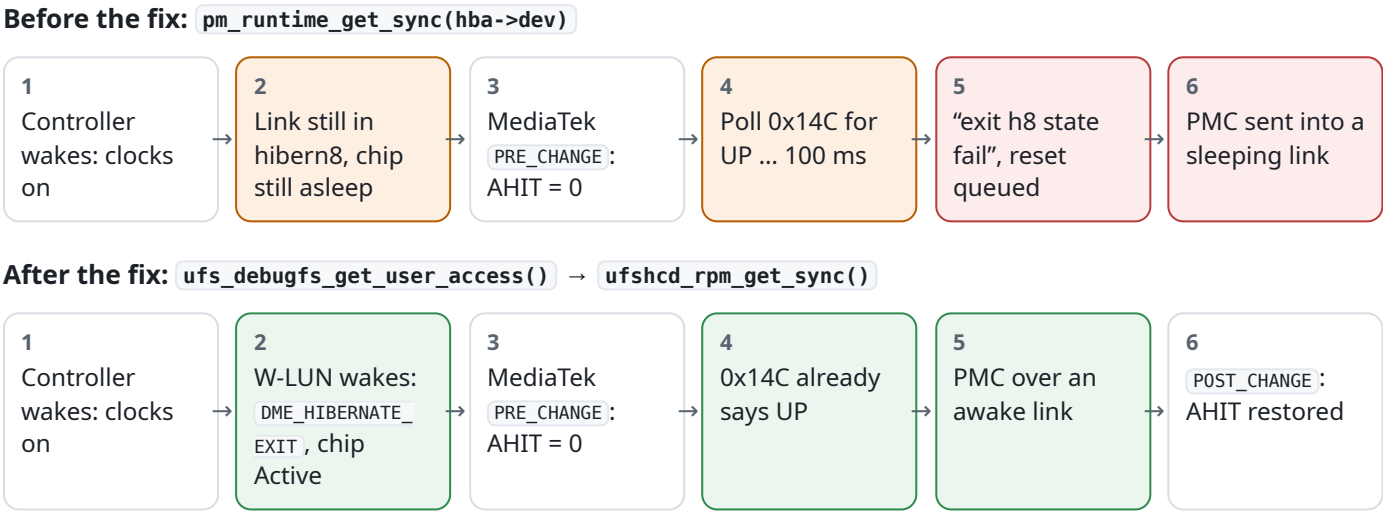


Figure 4. The same write, before and after. The only difference is step 2.

5.4 One bug, seen from two places

Errors 1 and 2 are the same problem. Without MediaTek's hook, the PMC itself ran into the sleeping link and timed out. With the hook, the hook noticed the sleeping link first and complained.

5.5 Why it could look random

Within half a second of any disk access, the LUs, and so the W-LUN, were still awake, the link was UP, and the old line was enough. If the old code ever worked, that's why.

5.6 The `#if 0` manual hibern8 exit

The ported code has a disabled block that called `ufshcd_uic_hibern8_exit()` directly. It aimed at the right symptom, but it would only have woken the link. The chip would still be in Sleep from START STOP UNIT, and the core's records would still say the W-LUN is suspended and the link is in hibern8, because `ufshcd_uic_hibern8_exit()` doesn't update `hba->uic_link_state`; its callers do (`ufshcd.c:4413`). The next real wake-up would then most likely send `DME_HIBERNATE_EXIT` to a link that's already awake, and fail. Waking the W-LUN does all three things, in the order the core expects.

Qualcomm's original also enters and exits hibern8 by hand, but for a different reason: around its clock switch, so that auto-hibern8 can't race it (`ufs-debugfs.c:1237` in msm-4.19).

6 The fix, and the other changes

6.1 The one line that mattered

Before (as ported from Qualcomm msm-4.19)

```
pm_runtime_get_sync(hba->dev);
ufshcd_hold(hba);
...
out:
...
ufshcd_release(hba);
pm_runtime_put_sync(hba->dev);
```

After (what runs now)

```
ret = ufs_debugfs_get_user_access(hba);
if (ret)
        return ret;
ufshcd_hold(hba);
...
out:
...
ufshcd_release(hba);
ufs_debugfs_put_user_access(hba);
```

The helper was already in the 6.6 file being edited:

```
static int ufs_debugfs_get_user_access(struct ufs_hba *hba)
__acquires(&hba->host_sem)
{
        down(&hba->host_sem);
        if (!ufshcd_is_user_access_allowed(hba)) {
                up(&hba->host_sem);
                return -EBUSY;
        }
        ufshcd_rpm_get_sync(hba);
        return 0;
}

static void ufs_debugfs_put_user_access(struct ufs_hba *hba)
__releases(&hba->host_sem)
{
        ufshcd_rpm_put_sync(hba);
        up(&hba->host_sem);
}
```

ufs-debugfs.c:68

- **`down(&hba->host_sem)`** takes the UFS core's “one big job at a time” semaphore. The error handler, system suspend, shutdown, the boot-time scan and the core's own sysfs writes all take it (`ufshcd.c` lines 6561, 10067, 10309, 9039, 1669), so the PMC can't collide with any of them.
- **`ufshcd_is_user_access_allowed()`** refuses during shutdown (it returns `!hba->shutting_down`).
- **`ufshcd_rpm_get_sync(hba)`** wakes the W-LUN, which by rule 1 wakes the controller first.

What happens now when you write `power_mode` after half a second of quiet:

1. The controller wakes: power supplies, clocks, interrupt.
2. The W-LUN wakes: `DME_HIBERNATE_EXIT` (link UP), START STOP UNIT (chip Active), AHIT timer back on.
3. MediaTek's `PRE_CHANGE` sets AHIT to 0; the link is already UP, so the check passes.
4. The PMC runs over an awake link. `POST_CHANGE` puts AHIT back.
5. The code releases everything, and half a second later the whole chip side sleeps again.

> **KEY POINT**
>
> MediaTek's own `clkscale_control` path never had this bug: it calls `ufshcd_rpm_get_sync()` before anything else (`ufs-mediatek.c:2322` and `2398`). The fix makes the path follow the same recipe.

6.2 The other changes in file 06

Changed 1: the mode digits

The ported code had a “failsafe” block that forced FAST mode, using a variable called `pwr`. There is no `pwr` in the ported function, so it wouldn't compile, and forcing FAST would turn every SLOW (PWM) request into a FAST one. (The block looks like it came from an early version of file 05, which had both problems.) File 06 replaces it with:

```
        if (desired_pwr_mode->pwr_rx == FASTAUTO_MODE)
                desired_pwr_mode->pwr_rx = FAST_MODE;
        else if (desired_pwr_mode->pwr_rx == SLOWAUTO_MODE)
                desired_pwr_mode->pwr_rx = SLOW_MODE;
        desired_pwr_mode->pwr_tx = desired_pwr_mode->pwr_rx;
```

Why map the AUTO modes at all? MediaTek's `ufs_mtk_pre_pwr_change()` (`ufs-mediatek.c:1908`) asks a shared core helper, `ufshcd_get_pwr_dev_param()`, to agree the final settings. That helper only treats `FAST_MODE` as high speed:

```
        if (dev_max->pwr_rx == FAST_MODE)
                is_dev_sup_hs = true;
        ...
        if (!is_dev_sup_hs && is_pltfrm_max_hs) {
                pr_info("%s: device doesn't support HS\n", __func__);
                return -ENOTSUPP;
```

ufshcd-pltfrm.c (the same helper in a newer tree, renamed `ufshcd_negotiate_pwr_params()`; same logic)

A FASTAUTO request looks to it like “the device doesn't support HS”. It returns `-ENOTSUPP` and leaves the result all zeros. MediaTek only prints `failed to determine capabilities` and carries on, so the PMC would go out asking for gear 0 on 0 lanes. SLOW requests are fine, because MediaTek switches the helper into PWM mode for them.

The same helper also gives TX and RX the same gear (the lower of the two requested) and the same mode, caps lanes at 2, and sets rate B. So `342211` becomes G3 in both directions.

Changed 2: don't report success after a queued reset

```
        /* AH8 disable failed in MTK's PRE_CHANGE: host reset queued */
        if (!ret && hba->ufshcd_state != UFSHCD_STATE_OPERATIONAL)
                ret = -EIO;
```

MediaTek's hook can fail, queue a reset and still let the PMC report success. This turns that case into an error. The states (`ufshcd.h:529`):

VALUE	STATE	MEANING HERE
0	`UFSHCD_STATE_RESET`	Being reset
1	`UFSHCD_STATE_OPERATIONAL`	All good
2	`UFSHCD_STATE_EH_SCHEDULED_NON_FATAL`	Error handler queued for a minor error
3	`UFSHCD_STATE_EH_SCHEDULED_FATAL`	Reset queued. This is what MediaTek's failed check sets.
4	`UFSHCD_STATE_ERROR`	Recovery failed

The debug print, `dev_err(hba->dev, "state=%d\n", hba->ufshcd_state)`, reads the same field.

The validate function

Qualcomm's original `ufsdbg_power_mode_validate()` allows gears up to G4 and only FAST or FASTAUTO. It rejects G5 and every SLOW request with “power parameters are not valid”. File 06 has a version that allows G1 to G5 and all four modes, if you need it.

6.3 The ported clock-scaling lines

What they try to do: slow the controller's clock when the link goes down to G1, and speed it up for anything higher, by calling `ufshcd_scale_clks()` before or after the PMC.

What actually runs on MT6991:

1. `ufshcd_scale_clks()` first calls MediaTek's `clk_scale_notify` hook, which returns at once (`ufs-mediatek.c:3172`). The core's clock scaling is switched off on this board: the device-tree flag `mediatek,ufs-clkscale-only-by-user` (`mt6991.dts:6768`) makes MediaTek skip setting `UFSHCD_CAP_CLK_SCALING` (`ufs-mediatek.c:1763`).
2. Then the core's `ufshcd_set_clk_freq()` walks its list of clocks. `ufs_sel` is still on that list, with 273 and 499.2 MHz from the device tree; MediaTek removes only the two *source* clocks from the list (`ufs-mediatek.c:1240`). So the core calls `clk_set_rate(ufs_sel, …)`, unless it believes the clock is already at that rate.
3. MediaTek's own path makes the same switch differently (`_ufs_mtk_clk_scale()`, `ufs-mediatek.c:3039`). It raises vcore to at least 0.65 V *before* moving `ufs_sel` to the fast source with `clk_set_parent()`, and lowers vcore only *after* moving to the slow source. The core's path never touches vcore.

Is that safe? Slowing a clock down is always safe. Speeding it up to 499.2 MHz is safe only if vcore is already high enough. After `echo 4 > clkscale_control`, MediaTek has raised vcore, and the lock stops anyone lowering it, so it's fine. The risky combination is `echo 1` or `echo 3` (MediaTek drops vcore to its minimum) followed by a G5 request through `power_mode`.

Two things not checked. Whether `clk_set_rate()` really moves this mux depends on MediaTek's clock driver, which wasn't read. And `ufshcd_scale_clks()` is `static` in the stock android15-6.6 `ufshcd.c` (line 1128); since the build links, the board's tree must make it visible, probably as part of the port.

> **CORRECTION**
>
> An earlier explanation said these lines were harmless because MediaTek's clock-scale hook does nothing. The hook does nothing, but the core still calls `clk_set_rate()` on `ufs_sel`. The conclusion stands (safe after `echo 4`; don't mix with 1 or 3), but the reason given was incomplete.

6.4 `clkscale_control`, and why `echo 4` first

MediaTek's sysfs file takes six values. This is its own help text (`ufs-mediatek-sysfs.c:688`), with what each value does on the board:

VALUE	HELP TEXT	ON THE BOARD
0	free run	Refused: needs the core's clock scaling, which is off (“only by user”)
1	scale down	G4 and the 273 MHz clock (`min_gear = UFS_HS_G4`), vcore lowered
2	scale up	Max gear and 499.2 MHz. The value at boot.
3	scale down and forbid change	Like 1, then locked
4	scale up and forbid change	Like 2, then locked
5	allow change and free run	Removes the lock

What `echo 4` does, through `clkscale_control_store()` (`ufs-mediatek-sysfs.c:710`), `ufs_mtk_dynamic_clock_scaling()` (`ufs-mediatek.c:2364`) and `ufs_mtk_config_pwr_mode()` (`ufs-mediatek.c:2271`):

1. Wake the W-LUN (`ufshcd_rpm_get_sync()`), block new commands, hold the clocks, wait for the doorbell to clear.
2. Raise vcore, ask the CPUs for low wake-up latency (a PM QoS request), move `ufs_sel` to 499.2 MHz.
3. PMC to `max_gear`, then print `PMC Success! [RX, TX]: gear=…`.
4. Release everything, then set `clk_scale_forbid = true`.

Why it helps while you experiment:

- The clock and vcore are ready for G5.
- With the forbid flag set, every later call to `ufs_mtk_dynamic_clock_scaling()` returns at its first line. Its callers are writes to `clkscale_control` and the eye-monitor scan (`eyemon_scan()`). The store function accepts a `powerhal_set:` prefix, which means Android's PowerHAL writes this file at runtime. Without the lock, one of those writes could run its own PMC and replace the setting.

WATCH OUT

`echo 2` or `echo 4` will not take you back to max gear after you've changed the gear through `power_mode`. `ufs_mtk_dynamic_clock_scaling()` remembers its last request in a `static` variable, `scale_mode`. If that was already “scale up”, it returns without a PMC; it doesn't know `power_mode` moved the gear. Write the max mode through `power_mode`, or reboot. `echo 5` removes the lock.

7 How we got here, and where we went wrong

7.1 The path, in order

1. **The question.** The starting point: the board, `clkscale_control`, the ported debugfs code and the two errors, and a search for a ready-made fix.
2. **The search.** We went through upstream Linux, MediaTek's vendor trees and the patch archives. No patch fixes this exact failure. Two are related, and neither fixes it: `7dec5cedf7b2` ("scsi: ufs: core: Serialize AHIT register access between sysfs and host driver") and `f29ec85ac4fd` ("scsi: ufs: host: mediatek: Return error directly on idle wait timeout"). We downloaded MediaTek's MT6991 vendor code (OnePlus's public copy), the android15-6.6 core files and Qualcomm's msm-4.19 `ufs-debugfs.c`, so the three could be read side by side.
3. **One step at a time.** The first explanation was too compressed, so we slowed down and went through it a question at a time. What does disabling AH8 actually do? Must the link be up before a PMC? Why would the W-LUN's suspend have sent `DME_HIBERNATE_ENTER`? Why is there a separate step that powers UniPro down?
4. **Code to compare.** Files 01 to 05 were written into this folder, to open side by side in an editor.
5. **The flow.** The ported code's sequence: `pm_runtime_get_sync()`, hold, lock, block, doorbell, PMC, plus the ported clock lines. Its first step was the root cause: it woke `hba->dev`, not the W-LUN.
6. **The clock question.** Keep the ported clock-scaling lines, or copy MediaTek's? We settled on keeping them, setting `clkscale_control` to 4 first, and never mixing with 1 or 3.
7. **The ported code.** The code switched to `ufs_debugfs_get_user_access()` (from file 05) on 31 October. It was copied into file 06, with two more fixes marked.
8. **The test.** We rebuilt the kernel image, set `clkscale_control` to 4 and wrote `power_mode`: no `exit h8 state fail`. Then the lane tests passed too.

7.2 Corrections along the way

These statements changed during the investigation; knowing which helps in weighing the rest.

WHAT WAS SAID	WHAT'S ACTUALLY TRUE
Force every request to FAST mode.	Only the AUTO modes need mapping. Forcing FAST breaks SLOW (PWM) requests. (6.2)
First: the ported clock lines are dangerous. Then: they're harmless because MediaTek's hook does nothing.	They're safe after `echo 4`, but the core does call `clk_set_rate()` on `ufs_sel`. Don't mix them with 1 or 3. (6.3)
`down_write(&hba->lock)` is a problem.	That lock doesn't exist in 6.6. The 6.6 equivalent is `hba->clk_scaling_lock`.
Put the gear and lane function in MediaTek's vendor module (files 02, 03 and the patch).	The debugfs is in the core, which can't call vendor code or see `struct ufs_mtk_host`. Files 05 and 06 target the core. The vendor version is still there if you ever want a sysfs node; it was never compiled.

WHAT WAS SAID	WHAT'S ACTUALLY TRUE
An early file 05 used a variable named `pwr`.	The parameter is `desired_pwr_mode`. Fixed in 05 and 06.
The sequence was assumed to come straight from Qualcomm.	It came through a port for the MediaTek SoC, which took it from the Qualcomm SoC kernel.
“Nobody sends `DME_HIBERNATE_EXIT`.”	Imprecise. The W-LUN's wake-up sends it. It just never ran, because nothing woke the W-LUN.
“Since v5.14…”, which read as if 6.6 had decided it.	It changed in 5.14 (2021); 6.6 inherited it. (Part 4)
The link-state register was called `REG_UFS_PROBE`.	On MT6991 it's `REG_UFS_UFS_MMIO_OTSD_CTRL` (0x14C). `REG_UFS_PROBE` is the older chips' register. (2.3)

KEY POINT

The bug wasn't in any one file. Each piece was right on its own terms: Qualcomm's line for Linux 4.19, the core's 2021 change, MediaTek's AH8 check for the case it was written for. The bug lived in the gap between them. Reading all three side by side is what found it.

8 How we know it's fixed

8.1 Results so far

- With the fix, and `clkscale_control` at 4: no `exit h8 state fail`.
- Lane changes from 2 to 1 pass.

Two more checks turn that into proof. If you've already run them, skip ahead.

8.2 Test 1: recreate the old failure condition

The old failure needed the W-LUN asleep. But `echo 4` wakes the W-LUN for a moment (its first scale-up after boot does), so a write right after it could pass even with the old code. Make sure the W-LUN is asleep before you write:

> **TRY IT**
>
> ```
> W=/sys/bus/scsi/devices/0:0:0:49488/power/runtime_status   # the UFS Device W-LUN
> cat $W                  # repeat until it says: suspended
> cat $W; echo 112211 > /sys/kernel/debug/…/power_mode; dmesg | tail
> ```
>
> It passes if you see `suspended`, then no `exit h8 state fail`, and `state=1`. If `0:0:0:49488` isn't there, find it with `ls /sys/bus/scsi/devices | grep 49488`.

8.3 Test 2: did the gear really change?

“No error” isn't proof, for four reasons:

1. **The write never fails in the shell.** `ufsdbg_power_mode_write()` ends with `return cnt;` even when the PMC failed, so `echo` always looks successful. Only dmesg tells you.
2. **The success line in dmesg isn't from `power_mode`.** `PMC Success! [RX, TX]: gear=…` comes from MediaTek's `ufs_mtk_config_pwr_mode()`, that is, from `echo 4`. The core path prints nothing on success (`ufshcd_print_pwr_info()` uses `dev_dbg`, `ufshcd.c:697`).
3. **Asking for the current mode does nothing.** If the request equals the current settings, `ufshcd_change_power_mode()` returns 0 without sending a PMC (“power already configured”, `ufshcd.c:4566`). Right after `echo 4` the link is at max gear, so asking for max gear tests nothing.
4. **`cat power_mode` shows what the core believes.** It prints `hba->pwr_info`, which the core updates only after a successful PMC (`ufshcd.c:4638`). That's good evidence, but it's still software reporting on itself.

The physical check: the wire sets a speed limit. Whatever the software says, data can't move faster than the lanes carry it. The ceiling is bits per second × 0.8 (for 8b/10b) × number of lanes ÷ 8 bits per byte:

GEAR	BITS/S PER LANE	DATA PER LANE	CEILING, 2 LANES
HS-G1B	1,457.6 M	146 MB/s	~290 MB/s

GEAR	BITS/S PER LANE	DATA PER LANE	CEILING, 2 LANES
HS-G3B	5,830.4 M	583 MB/s	~1.17 GB/s
HS-G4B	11,660.8 M	1,166 MB/s	~2.3 GB/s
HS-G5B	23,321.6 M	~2,870 MB/s (128b/130b)	~5.7 GB/s

These are ceilings before protocol overhead; real numbers land below them.

TRY IT

Go to G1 (`112211`), then:

```
echo 3 > /proc/sys/vm/drop_caches     # every run, or the read comes from RAM
dd if=/dev/block/sda of=/dev/null bs=1M count=1024
```

At G1, anything above about 290 MB/s means the link isn't at G1, because that's physically impossible. Then run the same at max gear and compare. Use G1 rather than G3: a simple `dd` may not fill a G3 link, so G3 and max gear could show similar numbers, while G1 is an unmistakable drop.

8.4 Reading the `state=` print

`state=1` (OPERATIONAL) means all good. `state=3` (EH_SCHEDULED_FATAL) means MediaTek's link check failed and a reset is queued: the old failure is back, so look at what woke (or didn't wake) the W-LUN.

9 Open items

- Run tests 1 and 2, if you haven't.
- Make `ufsdbg_power_mode_write()` return the real error instead of `cnt` after a failed PMC, so `echo` fails visibly.
- To go back to max gear, write it through `power_mode` or reboot; `echo 2` or `echo 4` won't do it (6.4).
- If the failure ever comes back, print register 0x14C inside `ufs_mtk_auto_hibern8_disable()` at the moment it fails (needs a vendor-module rebuild). Bits 31 to 24 reading 3 would confirm a link stuck in hibern8.
- The board's vendor tree and the OnePlus copy used here have the same vendor code, but check line numbers before relying on them.
- `ufsdbg_cfg_pwr_param()` writes `hba->max_pwr_info.info.pwr_rx = FAST_MODE`. That's core state the error handler uses to restore the link. It's harmless today, because the chip already reports FAST there; don't write anything else into it.

10 Glossary

AH8 (auto-hibern8)

The controller puts the link into hibern8 by itself after an idle time, and wakes it by itself when new commands arrive.

AHIT

The controller register (offset 0x18) that holds the AH8 idle time. 0 means off.

`clk_scale_forbid`

MediaTek flag set by `clkscale_control` values 3 and 4. It blocks MediaTek's own clock and gear changes until value 5 clears it.

Clock gating

Switching the controller's clocks off during short idle gaps (at least 10 ms on MediaTek). Separate from runtime PM. `ufshcd_hold()` blocks it.

DME

The part of UniPro that takes management commands from the driver: `DME_GET`, `DME_SET`, `DME_HIBERNATE_ENTER`, `DME_HIBERNATE_EXIT`.

Doorbell

Controller register (offset 0x58) with one bit per command slot. The driver sets a bit to start a command; the controller clears it when the command is done.

EH (error handler)

The core's recovery work. It resets the controller and restarts the link.

Gear

Bits per second per lane. HS-G1 to G5 are the fast family; the PWM gears are the slow one.

GKI

Google's Generic Kernel Image, the common kernel that holds the UFS core.

`hba`

“Host bus adapter”: the core's `struct ufs_hba`, describing one controller. `hba->dev` is the controller's `struct device`.

Hibern8

The link's deepest sleep that keeps its settings.

`host_sem`

The core's semaphore that keeps user requests, error handling, system suspend and shutdown from overlapping.

Lane

One pair of wires carrying data in one direction.

LU

Logical unit: a readable, writable slice of the chip (`sda`, `sdb`, …).

M-PHY

The electrical layer: the wires, their voltages, and recovering bits and timing from them.

PMC

Power mode change: switching gear, lanes or mode.

Rate A, rate B

The two base frequencies for HS gears. MediaTek uses B.

Runtime PM

Linux's per-device sleep and wake while the system is running.

SSU (START STOP UNIT)

The SCSI command that sets the chip's power mode: Active, Sleep, PowerDown.

UIC command

A command for the link layer, written to controller register 0x90.

UniPro

The link layer: frames, acknowledgements, re-sending, speed and power state.

UPIU

The packet format that carries SCSI commands over UFS.

vcore

The SoC's core supply voltage. The 499.2 MHz `ufs_sel` source needs at least 0.65 V.

vops

The table of vendor hooks that the core calls.

W-LUN

Well-known logical unit. The UFS Device W-LUN (`0xD0`, shown by Linux as `0:0:0:49488`) stands for the whole chip.

11 Files in this folder

`~/Personal_Study/kernel/mt6991_ufs_gear_lane/`

FILE	WHAT IT IS
`01_mtk_reference.c`	MediaTek's own `ufs_mtk_config_pwr_mode()`, with its steps marked [1] to [8]. The correct recipe, for reference.
`02_ufs_mtk_config_gear_lane.c`	A gear and lane function for the vendor module. Alternative path; not compiled.
`03_gear_lane_sysfs.c`	A sysfs file that calls 02.
`0001-ufs-mediatek-add-gear_lane-sysfs.patch`	02 and 03 as a patch for the vendor tree. Applies to the OnePlus copy; not compiled.
`vendor_6.6/`, `patched_6.6/`	The three MediaTek files before and after that patch.
`04_qcom_msm-4.19_ufsdbg_config_pwr_mode.c`	Qualcomm's original, which was ported to the MediaTek SoC.
`05_core_ufs-debugfs_config_pwr_mode.c`	A core version built on `ufs_debugfs_get_user_access()`.
`06_ported_ufsdbg_config_pwr_mode_fixed.c`	The ported function, with the fixes marked `CHANGED`.
`UFS_gear_lane_fix_explained.html` and `.pdf`	This document.

12 References

Commits

- `b294ff3e3449`: “scsi: ufs: core: Enable power management for wlun”. Asutosh Das and Can Guo; committed 10 May 2021; in Linux 5.14. The change at the heart of Part 4.
- `52abca64fd94`: “scsi: block: Do not accept any requests while suspended”. The block-layer change that turned the old design into a deadlock.
- `7dec5cedf7b2`: “scsi: ufs: core: Serialize AHIT register access between sysfs and host driver”. Related; doesn't fix this.
- `f29ec85ac4fd`: “scsi: ufs: host: mediatek: Return error directly on idle wait timeout”. Related; doesn't fix this.

Source copies read

- MediaTek vendor code: OnePlus's public MT6991 tree, `kernel/kernel_device_modules-6.6/drivers/ufs/` (`ufs-mediatek.c`, `ufs-mediatek.h`, `ufs-mediatek-sysfs.c`) and `mt6991.dts`.
- Android common kernel `android15-6.6`: `drivers/ufs/core/ufshcd.c`, `ufs-debugfs.c`, `ufshcd-priv.h`, `include/ufs/ufshcd.h`.
- Upstream Linux v5.13 and v5.14: `drivers/scsi/ufs/ufshcd.c`.
- Qualcomm msm-4.19 (LineageOS sm8250 kernel): `drivers/scsi/ufs/ufs-debugfs.c`.
- The ported code in `kernel-6.6/drivers/ufs/core/ufs-debugfs.c`, 31 October 2026.

Written with Claude Code, 31 October 2026, from a debugging session with Shrihari. All line numbers refer to the source copies listed above.